Examen restanta curs

* Required

1. Email *

2. Nume si Prenume (First name & last name) *

3. Numar matricol (faculty ID) *

Probleme (Problems)

4. [UP] Ce puteti spune despre executia urmatorului program? (What can you say about the execution of the next program? 1 point

```
#include <iostream>
class Test
{
    int a[5] = { 1,2,3,4,5 };
public:
    int* begin() { return a + 1; }
    int* end() { return a + 4; }
};

int main()
{
    Test t;
    for (auto& i : t)
    {
        std::cout << i << ",";
    }
    return 0;
}
```

Mark only one oval.

- 1,2,3,4,5,
- ✓ 2,3,4,
- 1,2,3,4,
- 2,3,4,5,
- 3,4,5,
- Eroare la executie datorita castului din int* la auto& / Runtime error due to invalid cast from int* to auto&

5. Ce va afisa urmatorul program daca executia este pe o masina de 32 de biti ? (What is the result of the execution of the next program, if run on a 32 bit machine ?) 1 point

```
1  #include <iostream>
2  class A {
3      void f1() {}
4  };
5  class B: public A {
6      void f2() {};
7      void f3() {};
8  };
9  class C : public A, private B {
10     void f4() {}
11     void f5() {}
12     void f6() {}
13 };
14
15 int main()
16 {
17     std::cout << sizeof(C);
18     return 0;
19 }
```

Mark only one oval.

- [x] 1
- [ ] 0
- [ ] 12
- [ ] 4
- [ ] 28
- [ ] 56

6. [UP] Ce se intampla la executia urmatorului program? What will happen when the next program is executed? 1 point

```
#include <iostream>
using namespace std;

class Masina {
protected:
    int km;
public:
    void SetKM() { km = 10; };
    int GetKM() { return km; };
};

class Dacia : public Masina {
public:
    void SetKM() { km = 20; };
};

int main() {
    Dacia d;
    d.SetKM();
    Masina* m = (Masina *) &d;
    m->SetKM();
    cout << d.GetKM();
    return 0;
}
```

Mark only one oval.

- ( ) programul compileaza, dar da eroare la executie (the code is compiled, however a runtime exception is triggered)
- ( ) programul nu compileaza (the code does not compile)
- (✓) afiseaza 10 (10 is printed)
- ( ) afiseaza 20 (20 is printed)

7. [UP] Dat fiind urmatorul scenariu, care este metoda corecta de apel a constructorului explicit din clasa parinte, atunci cand se apeleaza constructorul explicit din clasa copil? / Given the following code, what is the correct way to call the explicit constructor in the parent class, when the child explicit constructor is called? 1 point

```
class Parinte {
private:
    int x, y;
public:
    Parinte(int xx, int yy);
    ~Parinte();
};

class Copil : public Parinte {
private:
    double q;
public:
    Copil(double qq, int xx, int yy);
    ~Copil();
};
```

Mark only one oval.

- ( ) Nu se poate apela constructorul explicit al Parinte, prin apelul constructorului explicit din Copil / The call to explicit constructor in Parnte is not possible through the explicit constructor in Copil
- (✓) In constructorul explicit din Copil trebuie sa apelez constructorul explicit din Parinte / Inside the explicit constructor for Copil, I should make a call to explicit constructor of Parinte
- ( ) Se va apela automat constructorul implicit (Nu este necesar sa se apeleze constructorul explicit) / The implicit constructor in Parinte should automatically be called (I should not care about calling the explicit constructor)
- ( ) Constructorul explicit din Copil are o greseala: trebuie inlaturati parametrii xx si yy / The explicit constructor of Copil has an error: xx and yy parameters should be removed
- ( ) Constructorul pentru Parinte se apeleaza prin copy-constructorul pentru Copil, in faza de implementare / The constructor for Parinte will be called through the copy-constructor for Copil, at implementation time

8. [DE] Ce puteti spune despre urmatorul program? / What can you say about the following program? 1 point

```
#include <iostream>
using namespace std;

class TEST {
private:
    int membru;
public:
    TEST(int m) { membru = m; }
    ~TEST() { membru = 0; }
    int Get() { return membru; }
};

int main()
{
    TEST* mm = 0;
    mm = (TEST*)malloc(sizeof(TEST));
    new (mm) TEST(100);
    cout << mm->Get();
    mm->~TEST();
    free(mm);
    return 0;
}
```

Mark only one oval.

- [x] Executia este cea asteptata si afiseaza "100" / It works as expected and displays "100"
- [ ] Este o problema cu faptul ca se apeleaza manual destructorul / There is a problem with the fact that the destructor is manually called
- [ ] Nu pot folosi new daca folosesc malloc / new and malloc cannot be used both at the same time
- [ ] Este o problema cu faptul ca instanta mm se aloca cu malloc() / There is a problem with the allocation of mm, using malloc()
- [ ] Nu se poate apela manual destructorul / The destructor cannot be manually called

9. [DE] Ce puteti spune despre urmatorul program? / What can you say about the following program? 1 point

```
#include <iostream>

struct Number
{
    int n;
    Number(int value) : n(value) {}
    Number operator+ (const Number& number) { return Number(++n + number.n); }
};
int main() {
    Number n1 = 1;
    Number n2 = 2;
    Number n3 = n1 + n2;
    std::cout << n1.n << "," << n2.n << "," << n3.n;
    return 0;
}
```

Mark only one oval.

- [x] Printeaza 2,2,4 / Prints 2,2,4
- [ ] Nu compileaza pentru ca constructorul lui Number este privat / It does not compile due due to the fact that Number constructor is private
- [ ] Nu compileaza pentru ca data membru "n" nu poate fi accesata de catre std::cout / Does not compile because "n" data member from Number class can not accessed in the std::cout call
- [ ] Produce o exceptie la runtime cand se face suma intre n1 si n2 / n1+n2 produces a runtime exceptions
- [ ] Printeaza 1,2,3 / Prints 1,2,3
- [ ] Printeaza 2,2,3 / Prints 2,2,3

10. [UP] Care dintre urmatoarele expresii lamda capteaza variablele locale prin referinta ? / Witch one of the following lamba expression captures local variables with references ? 1 point

Mark only one oval.

- [x] [&](int x, int y) { return x+y; }
- [ ] [=](int x, int y) { return x+y; }
- [ ] [](int x, int y) { return x+y; }
- [ ] [*](int x, int y) { return x+y; }
- [ ] [this](int x, int y) { return x+y; }

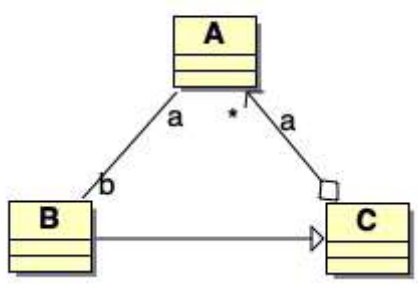
11. [DE] Cum se va comporta urmatorul program? / How will this application behave? 1 point

```cpp
#include <iostream>
struct A {
    virtual A* get() {
        return this;
    }
    void print() {
        std::cout << "A";
    }
};
struct B : A {
    B* get() override {
        return this;
    }
    void print() {
        std::cout << "B";
    }
};
int main() {
    auto a = (new B)->get();
    a->print();
}
```

Mark only one oval.

- ( ) eroare de compilare pentru ca functia virtuala get returneaza altceva in clasa B / compiler error due to the fact the virtual method get has a different return type in class B
- ( ) printeaza "A" / prints "A"
- (✓) printeaza "B" / prints "B"
- ( ) printeaza "AB" / prints "AB"
- ( ) printeaza "BA" / prints "BA"
- ( ) nu afiseaza nimic / doesn't print anything
- ( ) crasheaza cu exceptie neprinsa / crashes with unhandled exception

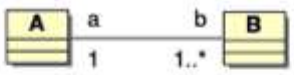
12. Care afirmații sunt adevărate despre următoarea diagramă de clase? What claims are true about the following class diagrams? 1 point



Mark only one oval.

- ( ) Există o relație de asociere unidirecțională de la A la C. / There is a unidirectional association relation from A to C.
- ( ) Există o relație de asociere bidirecțională între C și A. / There is a bidirectional association relation between C and A.
- (✓) Există o relație de agregare între C și A. / There is an aggregation relation between C and A.
- ( ) Există o relație de generalizare/specializare între A și B. / There is a generalization/specialization relation between A and B.
- ( ) Există o relație de generalizare/specializare între C și A. / There is a generalization/specialization relation between C and A.

13. [DE] Alegeți dintre descrierile C++ de mai jos implementează corect diagrama de clase din figură. / Choose the C++ that correctly implement the diagram. 1 point



1	`class A {`
	`  B b;`
	`  ...`
	`}`
		`class B : public A {`
		`  std::vector<A> a;`
		`  ...`
		`}`

2	`class A {`
	`  B b;`
	`  ...`
	`}`
		`class B {`
		`  std::vector<A> a;`
		`  ...`
		`}`

3	`class A {`
	`  std::list<B*> b;`
	`  ...`
	`}`
		`class B {`
		`  A* a;`
		`  ...`
		`}`

4	`class A {`
	`  B *b;`
	`  ...`
	`}`
		`class B {`
		`  std::list<A*> a;`
		`  ...`
		`}`

5	`class A {`
	`  std::list<B> b;`
	`  ...`
	`}`
		`class B {`
		`  A a;`
		`  ...`
		`}`

Mark only one oval.

- 1
- 2
- ✓ 3
- 4
- 5

This content is neither created nor endorsed by Google.

Google Forms